

EXPERIMENT 16

By Trishil Chauhan

Aim

To develop a menu-driven C program using **switch-case** and a separate **void user-defined function** for each menu option. Each function reads the positive integer itself, processes its digits, and displays the required result.

Operations

1. Check whether digits are in increasing order from left to right.
2. Check whether digits are in decreasing order from left to right.
3. Find the largest and smallest digit.
4. Find the second largest and second smallest **distinct** digit.

Note: Increasing/decreasing is treated as strict order. For example, 1237 is increasing and 8641 is decreasing. Repeated equal adjacent digits do not satisfy strict order.

Algorithm

Main algorithm

1. Start.
2. Display four menu options.
3. Read the user's choice.
4. Use switch-case to call exactly one void function: `increasing()`, `decreasing()`, `largestSmallest()`, or `secondLargestSmallest()`.
5. Each called function reads a positive integer, validates it, performs its digit operation, and prints the result.
6. If the choice is outside 1-4, display "Invalid choice".
7. Stop.

Digit-order method

Read the number as a digit string so digits can be compared directly from left to right. For increasing order, verify every digit is smaller than the next digit. For decreasing order, verify every digit is greater than the next digit.

Largest/smallest method

Repeatedly obtain each digit using $n \% 10$ and update minimum and maximum values.

Second largest/second smallest method

Mark which digits 0-9 occur. Scan upward to obtain the smallest and second smallest distinct digits, and downward to obtain the largest and second largest distinct digits. If fewer than two distinct digits exist, report that the second values do not exist.

Test Case Table

Test	Choice	Input	Expected result
1	1	1237	Digits are in increasing order
2	1	1327	Digits are not in increasing order
3	2	8641	Digits are in decreasing order
4	2	8624	Digits are not in decreasing order

Test	Choice	Input	Expected result
5	3	50729	Largest digit = 9; Smallest digit = 0
6	4	58391	Second largest = 8; Second smallest = 3
7	4	99820	Second largest = 8; Second smallest = 2
8	4	7777	Second largest/smallest do not exist
9	5	-	Invalid choice

C Program

```
#include <stdio.h>
#include <string.h>

void increasing(void);
void decreasing(void);
void largestSmallest(void);
void secondLargestSmallest(void);

int main(void)
{
    int choice;

    printf("1. Check increasing order\n");
    printf("2. Check decreasing order\n");
    printf("3. Find largest and smallest digit\n");
    printf("4. Find second largest and second smallest digit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1: increasing(); break;
        case 2: decreasing(); break;
        case 3: largestSmallest(); break;
        case 4: secondLargestSmallest(); break;
        default: printf("Invalid choice\n");
    }
    return 0;
}

void increasing(void)
{
    char n[50];
    int i, flag = 1;

    printf("Enter a positive integer: ");
    scanf("%49s", n);

    if (n[0] == '-' || n[0] == '0') {
        printf("Please enter a positive integer.\n");
        return;
    }

    for (i = 0; n[i + 1] != '\0'; i++)
        if (n[i] >= n[i + 1]) {
            flag = 0;
            break;
        }

    if (flag)
        printf("Digits are in increasing order\n");
    else
        printf("Digits are not in increasing order\n");
}

void decreasing(void)
{
    char n[50];
    int i, flag = 1;

    printf("Enter a positive integer: ");
    scanf("%49s", n);

    if (n[0] == '-' || n[0] == '0') {
        printf("Please enter a positive integer.\n");
        return;
    }

    for (i = 0; n[i + 1] != '\0'; i++)
        if (n[i] <= n[i + 1]) {
            flag = 0;
            break;
        }

    if (flag)
        printf("Digits are in decreasing order\n");
    else
        printf("Digits are not in decreasing order\n");
}
```

```

void largestSmallest(void)
{
    long long n, temp;
    int digit, largest = 0, smallest = 9;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

    if (n <= 0) {
        printf("Please enter a positive integer.\n");
        return;
    }

    temp = n;
    while (temp > 0) {
        digit = temp % 10;
        if (digit > largest) largest = digit;
        if (digit < smallest) smallest = digit;
        temp /= 10;
    }

    printf("Largest digit = %d\n", largest);
    printf("Smallest digit = %d\n", smallest);
}

void secondLargestSmallest(void)
{
    long long n, temp;
    int present[10] = {0};
    int digit, i, count = 0;
    int smallest = -1, secondSmallest = -1;
    int largest = -1, secondLargest = -1;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

    if (n <= 0) {
        printf("Please enter a positive integer.\n");
        return;
    }

    temp = n;
    while (temp > 0) {
        present[temp % 10] = 1;
        temp /= 10;
    }

    for (i = 0; i <= 9; i++) {
        if (present[i]) {
            count++;
            if (smallest == -1) smallest = i;
            else if (secondSmallest == -1) secondSmallest = i;
        }
    }

    for (i = 9; i >= 0; i--) {
        if (present[i]) {
            if (largest == -1) largest = i;
            else if (secondLargest == -1) secondLargest = i;
        }
    }

    if (count < 2)
        printf("Second largest and second smallest do not exist\n");
    else {
        printf("Second largest digit = %d\n", secondLargest);
        printf("Second smallest digit = %d\n", secondSmallest);
    }
}

```

Compilation and Execution

Save as: experiment16.c

Compile: gcc experiment16.c -o experiment16

Execute: ./experiment16

Sample Executions

Test 1 - Increasing order

Enter your choice: 1

Enter a positive integer: 1237

Digits are in increasing order

Test 2 - Largest and smallest

Enter your choice: 3

Enter a positive integer: 50729

Largest digit = 9

Smallest digit = 0

Test 3 - Second largest and second smallest

Enter your choice: 4

Enter a positive integer: 58391

Second largest digit = 8

Second smallest digit = 3

Result

The menu-driven C program was prepared using switch-case and four separate void user-defined functions. The algorithm and test cases were prepared, and the program logic satisfies all four required digit-based operations.